

6 Densest At-Most- k Subgraph on Twitch Ego-Nets

Setup. We evaluate the learning-augmented densest at-most- k subgraph algorithm (Algorithm 3), which we denote LA-DAMKS, on 500 ego-nets from the Twitch Ego Nets collection [1], with $30 \leq n \leq 52$ (mean $n = 40.4$, mean $m = 136.9$) and $k = 15$. Edges are treated as undirected. The ground truth H^* is computed exactly by Dinkelbach’s method [2]: given the current best set with e edges on s vertices, we solve the integer program

$$\max s \sum_{uv \in E} y_{uv} - e \sum_{v \in V} x_v \quad \text{s.t.} \quad y_{uv} \leq x_u, y_{uv} \leq x_v, 1 \leq \sum_v x_v \leq k, x \in \{0, 1\}^V,$$

strengthened with the valid inequalities $\sum_{u \in N(v)} y_{uv} \leq \min\{\sum_w x_w - x_v, (k - 1)x_v\}$, using HiGHS [3]. All coefficients are integers, so a positive optimum yields a strictly denser set, and an optimum of 0 certifies that the current set is optimal. All 500 instances are solved to proven optimality, with $5 \leq |H^*| \leq 15$.

Models and evaluation. A random forest (100 trees) predicts $p(v) \sim \mathbf{1}[v \in H^*]$ from five per-vertex features: degree, average neighbor degree, number of vertices, core number, and local clustering coefficient. Following Lemma 3.1, the predicted set is $S = \{v : p(v) \geq 1/2\}$. LA-DAMKS needs ε , which we estimate as the mean of $|S \triangle H^*|/|H^*|$ on a held-out 20% calibration split of the training graphs; this gives $\hat{\varepsilon} \in [0.44, 0.48]$ across folds. We use 5-fold cross-validation over graphs, so every graph is evaluated by a model that has not seen it. The main baseline, *predictor only*, returns S itself. We also report top- k , the k vertices with the largest $p(v)$, which is always feasible.

Results. Table 1 reports the ratio $d(\cdot)/d(H^*)$. The predictor is weak: its mean normalized error is $|S \triangle H^*|/|H^*| = 0.48$, far outside the small- ε regime of Theorem 3.4. Moreover, S is infeasible ($|S| > k$) on 205 of the 500 graphs. Even so, LA-DAMKS reaches a mean ratio of 0.937 (median 0.996) and is exactly optimal on 48.4% of the graphs. Predictor only reaches 0.817 even when its infeasible outputs are scored at face value. Restricted to graphs where S is feasible, it reaches only 0.705. Per graph, LA-DAMKS beats predictor only on 387 graphs, ties on 7 and loses on 106; in 86 of those losses, S was infeasible.

method	mean	median	min	≥ 0.95	optimal
LA-DAMKS ($\hat{\varepsilon}$)	0.937	0.996	0.250	74.6%	48.4%
predictor only, at face value	0.817	0.907	0.000	35.0%	18.2%
predictor only, $ S \leq k$ ($n=295$)	0.705	0.810	0.000	14.2%	2.7%
top- k by $p(v)$	0.894	0.913	0.532	27.4%	6.6%

Table 1: Ratio $d(\cdot)/d(H^*)$ on 500 Twitch ego-nets ($k = 15$, 5-fold cross-validation). “At face value” also counts the 205 graphs where $|S| > k$, so it overstates that baseline.

	mean	median	90th pct	max	total
exact ILP	213 ms	17.9 ms	617 ms	4480 ms	107 s
LA-DAMKS (end-to-end)	0.289 ms	0.290 ms	0.414 ms	1.06 ms	0.144 s

Table 2: Time per graph over the 500 graphs, single-threaded. The time for LA-DAMKS includes feature computation and forest inference but excludes the one-time training of the forest; the algorithm alone takes 0.009 ms on average.

The expansion step is what drives the gain. With ε fixed rather than estimated, the mean ratio increases monotonically from 0.813 at $\varepsilon = 0$ (only trimming S to k) to 0.891, 0.923, 0.938 and 0.950 at $\varepsilon = 0.1, 0.3, 0.5, 0.6$ (Figure 1b). Overestimating ε is therefore inexpensive here: adding more vertices with many edges into S costs little, because trimming removes the weakly attached ones.

Running time. Table 2 compares LA-DAMKS with computing H^* exactly by the integer program. Both run single-threaded on the same machine. LA-DAMKS is $74\times$ faster per graph (median), at least $19.5\times$ faster on every graph, and $738\times$ faster in total. The integer program’s cost grows quickly with graph size and has a heavy tail: its mean time rises from 82 ms for $n \in [30, 39]$ to 417 ms for $n \in [47, 52]$. Over the same range, LA-DAMKS stays between 0.24 and 0.34 ms with no loss in quality (mean ratio 0.940 and 0.948 respectively).

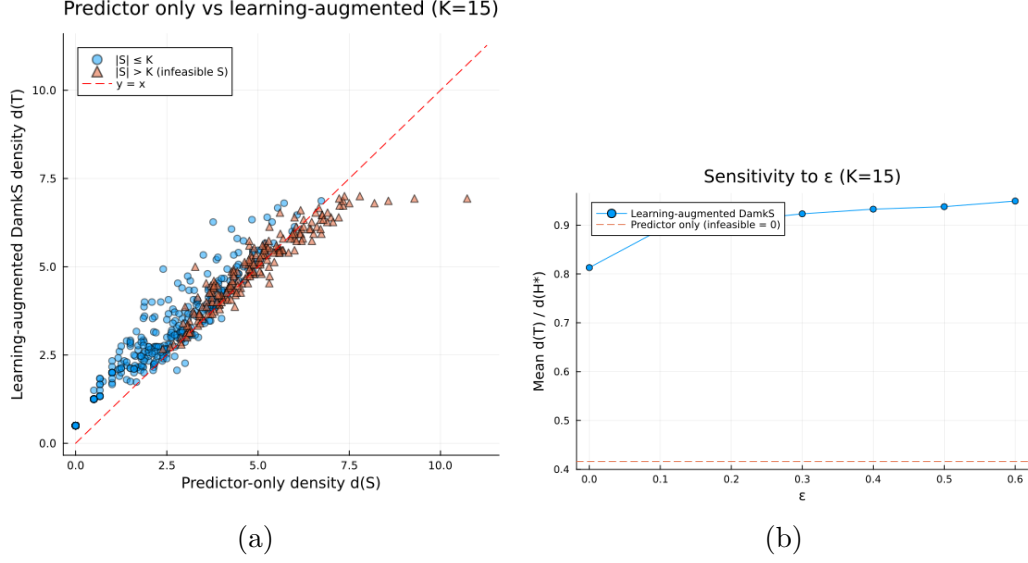


Figure 1: (a) Per-graph density of predictor only vs. LA-DAMKS. Triangles mark graphs with infeasible S . (b) Mean ratio of LA-DAMKS for a fixed ϵ .

References

- [1] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate Club: An API oriented open-source Python framework for unsupervised learning on graphs. In CIKM, pages 3125–3132. ACM, 2020.
- [2] Werner Dinkelbach. On nonlinear fractional programming. Management Science, 13(7):492–498, 1967.
- [3] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. Mathematical Programming Computation, 10(1):119–142, 2018.